HEADSHOT.io

Complete Browser Game Recreation Brief

Google AI Studio Master Prompt

Product name confirmed: HEADSHOT / HEADSHOT.io

A copy-paste-ready, implementation-oriented specification for recreating the fast, low-poly browser FPS experience: mechanics, weapons, modes, maps, UI, HUD, economy, audio, controls, accessibility and quality checks.

Reference image descriptions are included in the prompt. (The earlier image attachments were not retained in the workspace, so this rebuilt PDF contains the complete detailed prompt without embedded screenshots.)

COPY-PASTE MASTER PROMPT

You are a senior browser-game engineer, gameplay designer, 3D artist, UI designer, and QA engineer. Build a polished, playable, single-page browser first-person shooter named **HEADSHOT.io**. It should closely reproduce the fast, bright, low-poly, arcade tactical FPS experience shown in the attached reference images and described below, while using original code, original procedural art, and the new HEADSHOT identity. Do not copy the DEADSHOT logo, exact proprietary assets, or claim to be the original service. Deliver a complete runnable project, not a mockup, design document, or disconnected code snippets.

1. Non-negotiable output and engineering requirements

1. Create the full working game with the requested files. Use Three.js for 3D, vanilla JavaScript ES modules (or the project's existing framework if one is already established), HTML and CSS for interface, and no external model, texture, sound, or image dependencies. Generate all game geometry, material variation, textures, particles, and audio procedurally. Use a local vendored Three.js package if available; otherwise use the environment's supported dependency mechanism. Do not reference assets from a remote site.

2. The game must launch from a normal local HTTP server, render in a modern desktop browser, and also provide responsive touch controls for mobile. Start at the menu; Play enters a real playable match. Every visible button must do something. Persist profile, settings, unlocked cosmetic inventory, challenge progress, and statistics in `localStorage`, with safe defaults and versioned migration.

3. Organize code into clear modules: app/bootstrap and match lifecycle; config for weapons, modes, maps, economy and challenges; engine for input, physics, collision/raycast, procedural geometry, audio, profile; game for actors, bots, weapons, objectives and match rules; UI for menu, HUD, shop, locker, results and settings. Keep tuning numbers in configuration rather than burying them in rendering code.

4. Build a stable fixed-step simulation (target 60 updates/sec with capped catch-up) and independent render loop. Handle resize, tab visibility, pointer-lock loss, pause/resume, audio unlock on first user gesture, and cleanup when switching screens. Avoid runaway allocations in the frame loop. Include meaningful comments for non-obvious mechanics and a concise README with run instructions and controls.

5. Verify with a checklist: menu -> match -> kill/death -> respawn -> scoreboard -> match result -> menu; test every mode, map, weapon, input method, setting, economy interaction, and persistence. No dead links, placeholder buttons, console errors, blank screens, or fake "multiplayer" claims.

2. Art direction and presentation

Create an original HEADSHOT visual identity: compact uppercase wordmark, strong readable sans-serif typography, charcoal/navy interface panels, high-contrast white text, electric cyan/blue accents for allies and selection, orange/red for enemies and warnings, subtle bevels/glows, restrained motion, and crisp arcade readability. Establish clear visual hierarchy and consistent spacing. The 3D world is stylized low-poly, not photorealistic: readable silhouettes, simple bevelled shapes, clean flat-shaded surfaces, deliberate palette blocks, directional light, ambient fill, fog/sky gradient, and high-contrast landmarks. Keep the center of the screen uncluttered. Provide quality settings and reduced-motion option.

Reference image guide (eight authentic reference captures are included with this brief):

• **01** character/class promotional render and circular logo composition: use only as guidance for readable two-character/class presentation, not as an asset or logo to copy.

• **02** industrial courtyard splash: use for scale, industrial palette and action composition.

• **03** menu/lobby: reproduce the information hierarchy—brand, primary Play, mode/map or private-match controls, navigation to settings/shop/locker, challenge cards and a 3D operator showroom.

• **04** Refinery gameplay HUD: study centered first-person framing, crosshair, weapon/ammo, health, slide prompt, chat and contextual controls.

• **05** Vineyard ADS: sight picture, objective score/timer, touch controls and warm Mediterranean map palette.

• **06** Forest sniper: long sight lines, low-light sunset palette, scoped/hip-fire weapon framing.

• **07** Neo Tokyo SMG: night neon, lanterns and compact urban combat readability.

• **08** post-match screen: victory/defeat banner, team score, sortable player stats, weekly challenge card, replay and chat.

Reference files are `HEADSHOT_reference_images/01_official_classes_and_logo.png` through `HEADSHOT_reference_images/08_post_match_victory_scoreboard.png`. If the platform cannot read local files, use these written descriptions and still implement the game.

3. Core match loop, camera, controls

The player spawns into a first-person match, fights human-like bots, scores through kills/objectives, dies and respawns quickly, and sees a round-end results screen. Provide quick play and configurable private/custom match. The default mode is Team Deathmatch. Add a clear map/mode loading transition, countdown, pause overlay, match timer, scoreboard, kill feed, death/respawn feedback, round-end presentation and return-to-lobby route.

Desktop defaults: WASD move; mouse look; Space jump; Shift slide/crouch; left mouse fire; right mouse ADS; L alternate ADS; R reload; F weapon inspect; Tab scoreboard (hold); Enter chat; Esc or ~ pause. Support rebindable keyboard/mouse actions, pointer lock, sensitivity, separate ADS and scope multipliers, invert Y, FOV, and left-handed weapon presentation. Never trap the pointer before an explicit Play/Click to Play gesture. Include on-screen control hints for new players.

Player camera eye height is about 1.62 world units standing. Use a capsule-like actor with configurable standing height ~1.8, crouch ~1.2 and slide ~1.0. First-person viewmodel should show hands/arms and a procedural weapon with recoil, sway, ADS transition, reload, inspect, jump bob and landing response. Prevent camera clipping through walls. Add optional third-person character silhouettes for bots, built from procedural geometry, with head/torso/limbs and class-specific outfit colors. Team colors must remain recognizable and colorblind mode must add symbols/patterns, not rely on color alone.

4. Movement and advanced mechanics

Tune the movement to feel fast, responsive and skill-based. Starting targets (all configurable): walk speed ~7.2 units/sec; jump vertical impulse approximately 7.6; gravity ~22; slide boost ~11.5 units/sec for roughly 0.75 sec; slide cooldown ~1.0 sec beginning after the slide ends. Use acceleration/friction rather than instant velocity snapping. Collision must handle floors, walls, low cover, ramps and steps robustly. Keep movement authoritative in the local simulation; bots use the same collision rules.

• **Slide/crouch:** Shift while moving initiates a directional slide if ready. Sliding lowers camera/collider, increases speed, reduces weapon accuracy according to movement state, and ends on timer, collision, or loss of valid conditions. Hitting a wall at an angle greater than roughly 30 degrees cancels the slide instead of launching the player. Shift without movement or during cooldown becomes crouch. Releasing all WASD while still sliding snaps spread to crouch-level accuracy. Cooldown starts only after slide completion.

• **Slide jump / air strafe:** jumping during a slide preserves much of horizontal momentum. In air, directional input and mouse turning can bend velocity (Quake-like air acceleration) within limits. Show subtle white speed-edge streaks at high velocity; never obscure the reticle.

• **Bunny hop:** allow a jump just as the player lands to retain more momentum, with friction and acceleration caps to prevent runaway speed.

• **Shotgun jump:** firing the shotgun downward applies a vertical impulse; tune impulse so it is useful for traversal but not unlimited flight. It does not magically teleport and should provide recoil/audio feedback.

• **Crouch and accuracy:** movement, jumping, airborne state, crouch, slide and ADS each affect spread/recoil. Make states legible and consistent. ADS reduces spread and FOV smoothly; scoped sniper ADS uses a strong zoom and movement penalty.

• **Health and spawn protection:** player health regenerates after about four seconds without damage, at a restrained configurable rate; disable regen for relevant mutator. On respawn apply ~0.5 seconds of spawn invulnerability, clearly indicated and canceled when firing. Select spawn points that maximize distance from enemies and reject points with

an unobstructed enemy face-center line of sight; if all are compromised, choose the safest available. Never spawn inside geometry or directly in view where possible.

5. Weapons, classes and combat

Offer five selectable classes. Health, magazine, reserve ammunition, reload time, fire interval, damage, range falloff, spread, recoil, projectile type, ADS FOV, and movement penalty are configurable data. These are reference targets, not asserted hidden server values. Hitscan is suitable for ordinary bullets; shotgun uses thirteen individual pellets with distance falloff. Add head/body hit zones, generous but fair head hitboxes, friendly-fire policy (off by default), damage direction feedback, hit markers (normal, headshot, elimination), and a readable kill feed. Headshots add 50 score to a normal 100-point kill. Provide clear audio/visual distinction per weapon, reload and dry fire. Do not allow firing during reload; support interrupt rules consistently. Ammo is shown as current / reserve, with low-ammo cue.

```
| Class / weapon | Character silhouette | Reference target |
| Assault Rifle / M4A1-style | Bearded tactical male, dark cap and vest | 100 HP; 30-round magazine; automatic, approximately 450–600 RPM; r
| SMG / Vector-style | Agile female scout, grey top and olive sleeves | 100 HP; 40 rounds; automatic, ~720–900 RPM; holographic sight; near
| Sniper / AWP-style | Dark-suit agent with white cuffs | 85 HP; 3 rounds; bolt action ~50–60 RPM; high magnification scope, heavy scoped mo
| Shotgun / double barrel | Breacher/tactical operator | 85 HP; 2 shells; 13 pellets per shot; iron sights; per-pellet near ~16 head / 8 bod
| Semi-auto sniper / DMR | Marksman silhouette | 90–100 HP; 7–8 rounds; medium zoom; roughly 55–62 body damage, headshot capable of a one-sh
```

Make class selection available in lobby and after death/before respawn. Show weapon, health, role, ammo capacity and concise strengths/weaknesses. Ensure all five classes are viable in the bot sandbox; avoid an unbeatable sniper or dominant close-range shotgun.

6. Bots and room-code multiplayer extension

Since a functional network server is not supplied, implement a complete local single-player match against tactical bots, and architect a clean transport interface so a WebSocket authoritative server can later replace local actors. Do not present local bots as real online players. Menus may offer “Private room / Join room” as disabled-with-explanation or local simulated room-code flow only if clearly labeled as a prototype; do not claim actual remote matchmaking.

Bots have difficulty tiers (easy/normal/hard), reaction delay, aim error, awareness, aggression, objective preference, weapon-specific engagement ranges, and cooldowns. Give them simple state machines: spawn, seek objective/patrol, acquire target, take cover, peek/fire, reload, retreat when low, chase/collect a kill-confirmation token, and respawn. Use nav nodes/A* or waypoint graphs with line-of-sight and collision checks. Bots should strafe, use cover, jump/slide occasionally, avoid running into walls, and respect team boundaries. Add configurable names, team colors, scoreboards and ping-like display only as decorative local simulation; label bots appropriately in a local game.

7. Modes and scoring

Implement all six modes as distinct real rules with mode-specific HUD and end conditions:

1. **Free For All (FFA):** every player is enemy; kills score; leader indicator and individual ranking. Optionally use a 10-minute default, configurable.

2. **Team Deathmatch (TDM):** two teams, team score from eliminations; default target 50 kills or timer (configurable); display team score and player contribution.

3. **Hardpoint:** a capture hill rotates among fixed map locations about every 60 seconds. Team receives +1 point per second for each allied player inside; each player receives +10 personal score per second while contributing. Show active hill, countdown, contested/owned state and objective markers.

4. **Domination:** three neutral/capturable points A, B, C. Presence captures; owned points periodically score for their team. Show ownership, progress, distance markers and team score. Prevent progress when contested unless configuration specifies otherwise.

5. **Kill Confirmed (FFA):** an elimination drops a visible spinning translucent red low-poly pyramid token at the death location; collecting confirms and awards the kill score, denying/other collection rules are explicit and consistent. If no team, token colors/labels should communicate status.

6. **Team Kill Confirmed:** enemy elimination produces red token worth +20 on collection; friendly dropped blue token is worth +10 on recovery/deny. State the rule in mode help and show token feedback. Tokens expire after a configurable duration.

All modes need timer, target, tie handling, overtime only where sensible, pause/resume, respawn delay, and clean win/lose/draw state. Custom match allows map, mode, duration (5/10/20 minutes), bot count, difficulty and mutators: Headshots Only, One Shot Kills, Low Gravity, Infinite Ammo, Speed Boost, No Regen, Big Heads and Snipers Only. Explain mutators before launch. Ensure mutators change actual gameplay, not just labels.

8. Maps and world-building

Create six distinct, navigable maps from reusable procedural kits. Provide spawn points, nav routes, cover, alternate paths, elevated positions, fair sight lines and recognizable landmarks. Treat map sizes as approximate extents in world units, not rigid boundaries; use collision and kill-volume fall recovery. Add map preview thumbnails procedurally or via the reference screenshots only in this brief—not by hotlinking. Ensure objective points and spawns are distributed fairly across teams. Use original arrangement and details while matching these themes:

• **Factory (~96 × 72):** industrial hall and outer yard, shipping containers, loading bays, conveyor/pipe details, elevated catwalk, long but interrupted sniper lane. Steel grey, muted blue, safety yellow, rust orange. Interior/exterior transitions and multiple flank routes.

• **Refinery (~60 × 60, smallest):** compact process plant, tanks, red pipelines, yellow-railed catwalks and ladders/stairs. Add a fictional “PARAGON” plant sign as generic environmental dressing, not branding copied from game assets. Tight loops and vertical crossfire. Warm industrial lights.

• **Snowfall (~100 × 88):** snowy village with church, main plaza, narrow streets, pine-covered hills and a cave route. Snow caps on roofs and ledges, cold blue/white palette, warm window lights. Snowfall/fog as optional atmospheric weather; preserve visibility.

• **Forest (~100 × 100):** sunset canyon with mossy rocks, rolling terrain, two cave passages, bunker, lookout tower and trees. Long sightlines broken by rocks. Olive, deep green, amber and dusky orange. Avoid foliage blocking every route.

• **Vineyard (~100 × 84):** golden-hour Italian-inspired stone villa/winery, flag archway, central fountain, orderly vine rows and courtyard. Cream stone, terracotta roofs, rich green vines and warm sky. Optional subtle original acoustic ambience; no copyrighted music.

• **Neo Tokyo (~92 × 45):** compact nighttime Japanese-inspired neon street/subway corridor with orange lanterns, signage made from generic shapes/glyph-like decoration (avoid trademarked names), a white kei-style delivery vehicle, tracks/platform and overhead bridge. Purple/blue/cyan neon with warm orange accents. Keep the route readable and not overly dark.

Provide an original sky gradient, sun/moon, fog by distance, ambient occlusion approximation, shadows adjustable by quality, and small environmental motion (flags, drifting leaves/snow, steam, sparks) with low-cost particles. Add weather variations per outdoor map—clear, overcast, rain, fog, snow/blizzard—only if performance permits; weather must be cosmetic and never unfairly obscure enemies. Keep map lighting consistent with each theme. All geometry needs sensible colliders, walkable surfaces and out-of-bounds handling.

9. User interface and HUD

Implement every screen as a coherent navigable interface, not a single menu image:

Main menu/lobby: branded header; player profile/name and level; large Play/Quick Play; selected mode/map card; class selection; private match create/join code panel; clear navigation to Shop, Locker, Challenges, Leaderboard and Settings; daily and weekly challenge cards with progress/reward; currency display; a procedural 3D operator/weapon showroom with drag-to-rotate and subtle idle motion. Provide loading and error states.

HUD: center reticle changes with spread, movement, ADS and selected weapon. Include hit markers and distinct headshot/kill confirmation. Show health bar and numeric value, regeneration indication, current weapon, loaded/reserve ammo, reload state, score/team scores, timer, mode/objective, kill feed, damage direction arcs, teammate labels, enemy bars only when face-center line of sight is unobstructed, and slide/cooldown hint. Colorblind mode uses additional shape/icon distinctions. Tab-held scoreboard overlays without permanently obscuring action. Chat supports local preset messages or clearly local text; sanitize user-entered text. Keep HUD scalable and safe-area aware.

Scoreboard/results: sortable rows for name, score, kills, deaths, weapon, ping-like label (local simulation disclosed), with team grouping; show VICTORY/DEFEAT/DRAW, final score, XP, coins, challenge progress, return to lobby and Play Again. Results should match chosen mode and always allow exit.

Death/respawn: short readable death overlay, attacker/weapon if known, respawn timer, class switch and scoreboard access. Do not leave the player stuck if a bot or mode errors.

Mobile: landscape-first touch layout, left virtual joystick, right drag-look area, fire, ADS, jump, slide/crouch, reload, weapon/class interaction. Buttons must be large, movable only if implemented fully, opacity-adjustable and safe-area aware. Avoid accidental browser scrolling/zoom while playing; preserve accessible menu controls.

Accessibility: keyboard navigation in menus, visible focus, adequate contrast, text labels/icons, scalable UI, colorblind palette, reduced motion, mute control, no flashing strobe. Add tutorial overlay that can be dismissed and reopened.

10. Economy, cosmetics, challenges and profile

Create a self-contained cosmetic economy with no real-money purchases. Coins are earned from matches; reference match reward is `min(floor(score / 100), 25)`. Communicate that formula as a configurable target and show breakdown. Daily challenges: three tasks, +100 coins each. Weekly challenges: three tasks, +250 coins each. Track progress, completion, claim state and reset timestamps safely; in a fully offline demo, reset timers should be documented and not exploit local clock changes where possible.

Prize Wheel costs 500 coins per spin. Display probability tiers: Uncommon 50%, Rare 37%, Epic 10%, Legendary 3%; probabilities must sum to 100%. Award a cosmetic transparently with a clear result animation and no deceptive near-miss manipulation. Cosmetics are fictional procedural weapon finishes/charms and do not change stats. Five wear qualities: Flawless, Clean, Scuffed, Worn, Rugged. Generate wear in procedural canvas textures using scratches, speckle, faded patches and roughness variation; keep finish visible at weapon scale. Inventory has quantity/duplicate behavior clearly defined.

Trade-up consumes six eligible lower-tier items to award one item from the next tier, with confirmation, preview and no loss on cancel. Locker equips skins per weapon, shows ownership/rarity/wear, preview and reset-to-default. Shop/locker must be usable with keyboard and touch. Store coins, inventory, equipped cosmetics, stats, settings and challenge progress locally. Add “Reset local profile” with confirmation and explain it only affects this browser.

11. Procedural audio and visual feedback

Use Web Audio API only. On user gesture, create/resume AudioContext. Synthesize weapon-specific layered sounds: low thump, noisy transient/crack, filtered tail, mechanical action, reload clicks, shell/pellet variation, dry fire, hit confirmation, headshot/elimination stinger, footfalls by surface, jump/land, slide, objective capture, UI feedback and ambient loops. Use envelopes/noise/oscillators and per-class filter/pitch differences; avoid painful clipping. Spatialize approximate stereo pan and attenuate/low-pass distant sounds according to source distance and occlusion. Provide independent master/music/effects sliders and mute. No copyrighted audio samples or music. Optional Vineyard ambience must be original generated notes/texture and can be disabled.

Effects: muzzle flash, brief smoke, shell casings, impact sparks/dust, damage indicators, subtle camera shake with slider, recoil, tracer/projectile only where appropriate, kill-confirmation token spin/glow and objective capture pulses. Keep particle counts capped and pool reusable objects. Respect reduced-motion and low-quality modes.

12. Settings and tuning panel

Settings must work and persist: mouse sensitivity, ADS multiplier, scope multiplier, invert Y, FOV, graphics preset (low/medium/high), resolution scale, shadows, effects, master/music/SFX volume, crosshair shape/color/size/gap, static or dynamic reticle, left-handed viewmodel, colorblind mode, UI scale, camera shake, reduced motion, touch opacity, key bindings and reset defaults. Validate keybind conflicts and let users cancel/reset. Show a live crosshair preview and apply graphics changes immediately where safe. Never store sensitive data.

13. Robustness, quality and acceptance criteria

• Deliver runnable source, not just pseudocode. If you cannot finish every feature, prioritize a playable match, all six maps, five functional weapon classes, six modes, menus, responsive HUD and local persistence before cosmetic polish; disclose limitations honestly.

• The match must not depend on online services, account login, remote images, or hidden credentials. Do not imply real matchmaking or real multiplayer. Design a documented network adapter seam only.

• Use deterministic configurable spawn and objective logic; no spawn-in-wall or safe-zone exploit. Prevent firing through solid cover except explicitly configured penetrable foliage. The reference behavior includes leaves that can be wall-banged; implement an explicit material penetration table, with foliage penetrable at reduced damage and stone/metal blocking.

• Bots and actors need valid line-of-sight ray tests. Enemy overhead health bars appear only if face-center ray is clear. Spawn selection prefers farthest safe spawn with no enemy face-center LOS; if none is clear, choose the least exposed and retain ~0.5s protection.

• Ensure each map has spawn points, valid navigation, no inaccessible objective and a debug mode to visualize colliders, nav nodes, spawns, hitboxes and rays.

• Add unit or manual-test guidance for weapon damage, fire intervals, ammo, slide cooldown, shotgun jump, regeneration, spawn protection, mode score, challenge reward, wheel probabilities and trade-up. Clamp delta time after tab resume.

• Do not copy source-game code or assets. Use the reference screenshots solely to understand composition and broad gameplay language; HEADSHOT logo, UI details and procedural art should be original.

14. Suggested implementation sequence

1. Scaffold page and Three.js renderer, responsive canvas, camera, input and local profile.
2. Implement collision world, movement, first-person camera, weapon viewmodels, shooting/hit zones and bots.
3. Build reusable map kit and all six maps, then verify routes and spawns.
4. Implement six mode rules, scoring, objectives, kill tokens, respawn, scoreboard and results.
5. Build lobby/navigation, class picker, HUD, mobile controls, settings and accessibility.
6. Add challenges, coins, shop wheel, inventory, wear, trade-up and persistence.
7. Add synthesized audio, particles, environmental atmosphere, map previews and polish.
8. Run the acceptance checklist and fix regressions before calling the task complete.

15. Final response format

When finished, report: what is implemented; how to run it; controls; any known limitations (especially networking); tests/checks performed; and the principal files created. Do not claim feature completeness if anything is nonfunctional. Provide a concise change summary and ensure the project can be launched immediately.